



Tablouri bidimensionale

Parcurgeri · Împărțirea în zone · Simulări

Andrei Mihai Ciobanu

Algolymp Summer School

Declarare și citire

Parcurgeri

Tablouri pătratice — Diagonale

Împărțirea în zone

Transformări

Recapitulare

Resurse

Probleme de antrenament

Declarare și citire

Ce este o matrice?

Un **tablou bidimensional** (matrice) are elementele aranjate pe **linii** și **coloane**.

Fiecare element are doi indici: i (linia) și j (coloana). Notăm $a[i][j]$.

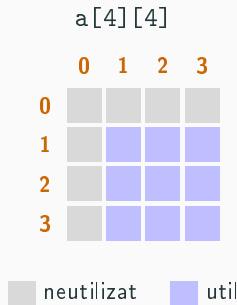
	1	2	3	4
1	3	7	2	9
2	1	5	8	4
3	6	0	3	1

Exemplu: $n = 3$ linii, $m = 4$ coloane. $a[2][3] = 8$.

Declarare în C++

```
1  const int NMAX = 105;  
2  int a[NMAX][NMAX]; // declarata global  
3  int n, m;           // dimensiunile curente
```

- **Declarare globală** — variabilele locale stau pe *stivă* ($\approx 1-8$ MB); globalele în segmentul de date, mult mai mare
- **Convenție: indexare de la 1** — declarăm $a[N+1][M+1]$ și lăsăm linia/coloana 0 neutilizate
- 1000×1000 de `int` ≈ 4 MB — alegeți NMAX cu grijă



Citire și afișare

```
1  cin >> n >> m;
2  for (int i = 1; i <= n; i++)
3      for (int j = 1; j <= m; j++)
4          cin >> a[i][j];
5
6  for (int i = 1; i <= n; i++) {
7      for (int j = 1; j <= m; j++)
8          cout << a[i][j] << ' ';
9      cout << '\n';
10 }
```

- Citim mai întâi n și m , apoi elementele **pe linii** (de sus în jos)
- La afișare, după fiecare linie scriem `'\n'` pentru rândul următor

Parcurgeri

Parcurgere pe linii și pe coloane

Pe linii (de sus în jos):

```
1  for (int i = 1; i <= n; i++)  
2      for (int j = 1; j <= m; j++)  
3          // prelucreaza a[i][j]
```

	1	2	3
1	1	2	3
2	4	5	6
3	7	8	9

Pe coloane (de la stânga la dreapta):

```
1  for (int j = 1; j <= m; j++)  
2      for (int i = 1; i <= n; i++)  
3          // prelucreaza a[i][j]
```

	1	2	3
1	1	4	7
2	2	5	8
3	3	6	9

- Complexitate: $O(n \cdot m)$ — fiecare element e vizitat exact o dată

Parcurgere pe o linie sau coloană fixă

Linia k fixată:

```
1 for (int j = 1; j <= m; j++)  
2     // a[k][j]
```

Toate elementele liniei k au același indice de linie.

- Complexitate: $O(m)$ pentru o linie, $O(n)$ pentru o coloană

Coloana k fixată:

```
1 for (int i = 1; i <= n; i++)  
2     // a[i][k]
```

Toate elementele coloanei k au același indice de coloană.

Tablouri pătratice — Diagonale

Ce sunt diagonalele?

Un tablou bidimensional este **pătratic** dacă $n = m$. Folosim o singură variabilă n pentru ambele dimensiuni.

Diagonala principală

De la $(1, 1)$ la (n, n) .

Condiție: $i = j$

	1	2	3	4
1	M	.	.	.
2	.	M	.	.
3	.	.	M	.
4	.	.	.	M

Diagonala secundară

De la $(1, n)$ la $(n, 1)$.

Condiție: $i + j = n + 1$

	1	2	3	4
1	.	.	.	S
2	.	.	S	.
3	.	S	.	.
4	S	.	.	.

Dacă n este impar, cele două diagonale au un element comun (centrul matricei).

Parcurgerea diagonalelor

```
1 // diagonala principala: conditia i == j
2 for (int i = 1; i <= n; i++)
3     // a[i][i]
4
5 // diagonala secundara: conditia i + j == n + 1
6 for (int i = 1; i <= n; i++)
7     // a[i][n + 1 - i]
```

- Ambele diagonale au n elemente $\rightarrow$ complexitate $O(n)$
- **Atenție:** la indexare de la 0 condiția diagonalei secundare devine $i + j = n - 1$, nu $n + 1$

Împărțirea în zone

Zone față de diagonala principală

	1	2	3	4
1	M	A	A	A
2	B	M	A	A
3	B	B	M	A
4	B	B	B	M

- **M** $i = j$: pe diagonală
- **A** $i < j$: deasupra diagonalei
- **B** $i > j$: sub diagonală

Condiția depinde **doar de indici**,
nu de valorile elementelor.

Zone față de diagonală secundară

Pentru $n = 4$ ($n + 1 = 5$):

	1	2	3	4
1	A	A	A	S
2	A	A	S	B
3	A	S	B	B
4	S	B	B	B

- **S** $i + j = n + 1$: pe diagonală
- **A** $i + j < n + 1$: deasupra
- **B** $i + j > n + 1$: sub diagonală

Atenție: la indexare de la 0, pragul este $n - 1$, nu $n + 1$.

Exemplu: suma elementelor din fiecare zonă

```
1 long long sp = 0; // suma deasupra diag. principale
2 long long sd = 0; // suma sub diagonala principala
3
4 for (int i = 1; i <= n; i++)
5     for (int j = 1; j <= n; j++) {
6         if (i < j) sp += a[i][j];
7         if (i > j) sd += a[i][j];
8     }
```

- Parcurgem matricea o singură dată: $O(n^2)$
- Putem combina condițiile pentru mai multe zone în aceeași buclă
- **Optimizare:** calculăm sumele *în timp ce citim* — nu e nevoie să stocăm matricea

Transformări

Transpusa matricei

Transpusa reflectă matricea față de diagonala principală:

$$b[j][i] = a[i][j].$$

```
1 // transpusa intr-o matrice auxiliara
2 int b[NMAX][NMAX];
3 for (int i = 1; i <= n; i++)
4     for (int j = 1; j <= m; j++)
5         b[j][i] = a[i][j];
6 // b are m linii si n coloane
```

Pentru matrice **pătratică**, transpusa se poate face în loc (*in-place*):

```
1 for (int i = 1; i <= n; i++)
2     for (int j = i + 1; j <= n; j++)
3         swap(a[i][j], a[j][i]);
```

Vertical (inversăm liniile):

```
1 for (int i = 1; i <= n / 2; i  
  ++)  
2   for (int j = 1; j <= m; j++)  
3     swap(a[i][j],  
4          a[n + 1 - i][j]);
```

Orizontal (inversăm coloanele):

```
1 for (int i = 1; i <= n; i++)  
2   for (int j = 1; j <= m / 2;  
      j++)  
3     swap(a[i][j],  
4          a[i][m + 1 - j]);
```

- Complexitate: $O(n \cdot m)$ pentru fiecare operație
- Rotire 90° la dreapta = transpusă + oglindire orizontală

Recapitulare

Recapitulare

Orice algoritm care parcurge **toată** matricea (parcurgere, sume pe zone, transpusă, oglindire etc.) are complexitate:

Matrice	Complexitate
$n \times m$ (dreptunghiulară)	$O(n \cdot m)$
$n \times n$ (pătratică)	$O(n^2)$

Excepții (parcurgere parțială):

- O linie sau coloană: $O(m)$ respectiv $O(n)$
- O diagonală: $O(n)$

Resurse

- **pbinfo.ro** — Tablouri bidimensionale
- **pbinfo.ro** — Declararea matricelor. Referirea elementelor
- **pbinfo.ro** — Parcurgerea tablourilor bidimensionale
- **pbinfo.ro** — Tablouri pătratice

Probleme de antrenament

Parcurgerea matricilor oarecare

- **Sum Even #996** (AlgoCode)
- **Algorithm Task #988** (AlgoCode)
- **Algorithm Task #1625** (AlgoCode)

Parcurgerea matricilor pătratice

- **Algorithm Task #1006** (AlgoCode)
- **Algorithm Task #1004** (AlgoCode)
- **Algorithm Task #999** (AlgoCode)

Generarea matricilor

- **Sequence Mat #963** (AlgoCode)
- **Sequence Mat #968** (AlgoCode)
- **Sequence Mat 17 #973** (AlgoCode)